

Artificial Intelligence Algorithms - Python Implementations

1. Alpha-Beta Pruning

```
def alpha_beta(node, depth, alpha, beta, maximizing_player, get_children, evaluate):
    if depth == 0 or not get_children(node):
        return evaluate(node)

    if maximizing_player:
        value = float('-inf')
        for child in get_children(node):
            value = max(value, alpha_beta(child, depth - 1, alpha, beta, False, get_children, evaluate))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for child in get_children(node):
            value = min(value, alpha_beta(child, depth - 1, alpha, beta, True, get_children, evaluate))
            beta = min(beta, value)
            if beta <= alpha:
                break
        return value

tree = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': [], 'F': [], 'G': []}
values = {'D': 3, 'E': 5, 'F': 6, 'G': 9}
get_children = lambda n: tree[n]
evaluate = lambda n: values.get(n, 0)
print("Alpha-Beta Result:", alpha_beta('A', 3, float('-inf'), float('inf'), True, get_children, evaluate))
```

2. Map Coloring using Backtracking (CSP)

```
def is_safe(region, color, assignment, neighbors):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def map_coloring(neighbors, colors, assignment={}):
    if len(assignment) == len(neighbors):
        return assignment

    region = [r for r in neighbors if r not in assignment][0]

    for color in colors:
        if is_safe(region, color, assignment, neighbors):
            assignment[region] = color
            result = map_coloring(neighbors, colors, assignment)
            if result:
                return result
            assignment.pop(region)
    return None

neighbors = {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'D'], 'D': ['B', 'C']}
colors = ['Red', 'Green', 'Blue']
print("Map Coloring Solution:", map_coloring(neighbors, colors))
```

3. Cryptarithmic Problem

```
import itertools

def solve_cryptarithm(word1, word2, result):
```

```

letters = set(word1 + word2 + result)
for perm in itertools.permutations('0123456789', len(letters)):
    table = dict(zip(letters, perm))
    if table[word1[0]] == '0' or table[word2[0]] == '0' or table[result[0]] == '0':
        continue
    n1 = int(''.join(table[c] for c in word1))
    n2 = int(''.join(table[c] for c in word2))
    n3 = int(''.join(table[c] for c in result))
    if n1 + n2 == n3:
        return {c: table[c] for c in letters}
return None

solution = solve_cryptarithm("SEND", "MORE", "MONEY")
print("Cryptarithm Solution:", solution)

```

4. Atomic Sentence Check

```

def is_atomic(sentence):
    logical_connectives = ['AND', 'OR', 'NOT', '=>', '<=>']
    for conn in logical_connectives:
        if conn in sentence:
            return False
    return True

sent1 = "Loves(John, Pizza)"
sent2 = "Loves(John, Pizza) AND Eats(Pizza)"
print(f"{sent1} → Atomic? {is_atomic(sent1)}")
print(f"{sent2} → Atomic? {is_atomic(sent2)}")

```

5. FOL Knowledge Base

```

class KnowledgeBase:
    def __init__(self):
        self.facts = set()
        self.rules = []

    def add_fact(self, fact):
        self.facts.add(fact)

    def add_rule(self, premise, conclusion):
        self.rules.append((premise, conclusion))

    def infer(self, query):
        if query in self.facts:
            return True
        for premise, conclusion in self.rules:
            if all(p in self.facts for p in premise):
                self.facts.add(conclusion)
                if conclusion == query:
                    return True
        return False

kb = KnowledgeBase()
kb.add_fact("Man(Socrates)")
kb.add_rule(["Man(Socrates)"], "Mortal(Socrates)")
print("Query: Mortal(Socrates)?", kb.infer("Mortal(Socrates)"))

```

6. Unification Algorithm

```

def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if theta == "failure":
        return "failure"
    elif x == y:
        return theta
    elif isinstance(x, str) and x.islower():

```

```

        return unify_var(x, y, theta)
    elif isinstance(y, str) and y.islower():
        return unify_var(y, x, theta)
    elif isinstance(x, tuple) and isinstance(y, tuple):
        if len(x) != len(y):
            return "failure"
        return unify(x[1:], y[1:], unify(x[0], y[0], theta))
    else:
        return "failure"

def unify_var(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)
    elif x in theta:
        return unify(var, theta[x], theta)
    else:
        theta[var] = x
        return theta

print("Unification Result:", unify(("Knows", "John", "x"), ("Knows", "John", "Mary")))

```